



Computer Science Competition State 2026 Programming Problem Set

I. General Notes

1. Do the problems in any order you like. They do not have to be done in order from 1 to 12.
2. All problems have a value of 60 points.
3. There is no extraneous input. All input is exactly as specified in the problem. Unless specified by the problem, integer inputs will not have leading zeros. Unless otherwise specified, your program should read to the end of file.
4. Your program should not print extraneous output. Follow the form exactly as given in the problem.
5. A penalty of 5 points will be assessed each time that an incorrect solution is submitted. This penalty will only be assessed if a solution is ultimately judged as correct.

II. Names of Problems

Number	Name
Problem 1	Cruz
Problem 2	AJ
Problem 3	Micah
Problem 4	Rio
Problem 5	Bish
Problem 6	Jackie
Problem 7	Bash
Problem 8	Steve
Problem 9	Bosh
Problem 10	Gustave
Problem 11	Verso
Problem 12	Maelle

1. Cruz

Program Name: Cruz.java

Input File: none

Cruz and his teammates are excited to be at the State Championship Meet and has created a Final Boss Screen.

Input: None

Output: The castle as below. The left-most character has zero spaces at the beginning of the line. There are no additional spaces at the end of each line. *There is a line of dashes (-) and numbers at the bottom which is NOT to be part of your output. Rather, it is there to help you correctly size the castle and position it for judging.*

Sample output:

[illegible]

1---5---0---5---0---5---0---5---0---5---0

2. AJ

Program Name: AJ.java

Input File: AJ.dat

AJ has recently joined the baseball team (as a manager). You need to help determine, based on which bases are taken, and what kind of hit it was (Fly, Grounder, Homer), the order in which to prioritize throwing to bases to minimize runs and maximize outs. If a hit is a fly, we can assume it will be caught, which means that you need to throw to the bases that are occupied (If a fly is caught, field runners have to tag up on their original base before continuing), and can ignore the runner from home to first since they are already out. If a homer is hit, then print out "Give Up", since there is nothing you can do. If a hit is a grounder, you need to throw to the base following the occupied bases (so someone is on third, throw to home, or someone on second, throw to third), and account for the runner from home to first. You should always prioritize outs closer to getting home, so always throw to the base closest to home first (home > third > second > first), and then in descending order from there. If there are no runners to be thrown out, output "None".

Input:

The input will begin with one integer, n ($0 < n \leq 1000$). Each test case will consist of 3 characters in a row (denoting first, second, and third base), either O or X, with X denoting that the base is taken, and O meaning it is open, followed by a space, followed by a string denoting what kind of hit it was (Fly, Grounder, Homer).

Output:

For each test case, output the prioritized order of bases to be thrown to, separated by spaces. The bases are as follows: 1st, 2nd, 3rd, Home

Sample input:

```
3
XOX Fly
OOX Grounder
XOO Grounder
```

Sample output:

```
3rd 1st
Home 1st
2nd 1st
```

3. Micah

Program Name: Micah.java

Input File: micah.dat

Micah wants to analyze the instructions for his team's automated robot and make sure the robot visits the expected waypoints on the field. The field of play is labeled as a two-dimensional grid. The robot always starts at the origin position, (0, 0), the center of the field. The instructions are listed with a direction (N, S, E, W) followed by a (positive) number of steps for the robot to take in that direction. N is in the positive y direction, S is in the negative y direction, E is in the positive x direction, and W is in the negative x direction. The field is a grid with x values in the range [-25, 25] and y values in the range [-25, 25]. There is a wall which will keep the robot from going off the field. If a robot runs into a wall, it just stops the current leg at the edge and goes on to the next leg.

Input: The first line contains an integer, T, which is the number of test cases ($1 \leq T \leq 10$). The next T groups of lines will begin with a single integer, N ($1 \leq N \leq 25$), representing the number of legs of the robot's journey. The following N lines will consist of a direction, D (N, S, E, W), and an integer, S ($0 \leq S \leq 25$), for the number of steps for the robot to take in the given direction.

Output: Each test case's output will begin and end with 25 #'s. Next, the output should report, on separate lines, each leg of the robot's journey with the coordinates of the ending location with an arrow (->) in between. Print a blank line after all the legs are displayed. The next line will be "Final location", a single space, and ": (x, y)" with x,y being the coordinates at the end of the robot's completed run. The next line will be "Distance", seven (7) spaces, and ": d" with d being the total number of steps the robot has taken. The next line will be "Bounds", nine (9) spaces, and ": x = [xMin, xMax], y = [yMin, yMax]" where xMin, xMax, yMin, and yMax are the extremes for the x and y coordinates. The next line will be "Commands", seven (7) spaces, and ": N=n S=s E=e W=w" where n, s, e, and w are the number of legs taken in each direction. Add a blank line after the closing line of #s.

Sample input:

```
1
6
N 5
E 3
S 2
E 4
W 1
N 6
```

Sample output:

```
#####
Leg 1 -> (0,5)
Leg 2 -> (3,5)
Leg 3 -> (3,3)
Leg 4 -> (7,3)
Leg 5 -> (6,3)
Leg 6 -> (6,9)

Final location : (6,9)
Distance      : 21
Bounds       : x = [0,7], y = [0,9]
Commands     : N=2 S=1 E=2 W=1
#####
```

4. Rio

Program Name: Rio.java

Input File: rio.dat

Rio's robotics team hosted a scrimmage with multiple fields, and the systems recorded match logs for each field. These log files each contains time-stamped entries for events such as queueing, start, score posted, timeout, and replay. Rio needs to merge them files together and generate a consistency report (detect any match that has START before QUEUED or POSTED before START) and detect any matches that had a replay. At end, Rio needs to report the time of the first event and the time of the last event. Sometimes one field can get ahead of the others (s) and a match can be moved from one field to another to keep things from getting too off schedule.

Note: when timestamps tie, preserve the original order (whichever was read in first); a match may appear on multiple fields; invalid matches should be listed once per invalid condition encountered.

Input: The first line in the data file is a single integer T ($0 < T \leq 10$) representing the number of test sets of data in the file. For each case, the first line will be an integer F ($1 \leq F \leq 5$) representing the number of fields in play. For each field, the first line will be the name of the field. The next line will be a single integer N representing the number of events that were logged. The following N ($1 < N \leq 50$) lines will be in the following format: HH:MM:SS MATCH_ID EVENT. Where MATCH_ID has no spaces, EVENT is one of the following: QUEUED, START, POSTED, TIMEOUT, or REPLAY.

Output: For each test case, begin and end the output with 14 '='s. Then, print the fully merged log in chronological order and numbered:

```
1. 08:00:00 ALPHA M1 QUEUED.
```

Next, print a blank line followed by a report of invalid and replayed matches separated by a blank line:

Invalid Matches:

```
YY -> START before QUEUED on field ALPHA
```

Replay Matches:

```
YY
```

In the above samples, YY is the match ID. If there are no matches in either or both categories, display NONE. If there are multiple matches in each category, print each on separate lines in chronological order.

Finally, print the times of the first and final events:

```
Time Range: 08:00:00 to 08:20:00
```

Separate each test set report with a single blank line.

Sample input:

```
1
2
ALPHA
4
08:00:00 M1 QUEUED
08:05:00 M1 START
08:12:00 M1 POSTED
08:20:00 M2 START
BETA
4
08:01:00 M2 QUEUED
08:06:00 M2 REPLAY
08:10:00 M2 POSTED
08:18:00 M2 START
```

Sample output:

```
=====
1. 08:00:00 ALPHA M1 QUEUED
2. 08:01:00 BETA M2 QUEUED
3. 08:05:00 ALPHA M1 START
4. 08:06:00 BETA M2 REPLAY
5. 08:10:00 BETA M2 POSTED
6. 08:12:00 ALPHA M1 POSTED
7. 08:18:00 BETA M2 START
8. 08:20:00 ALPHA M2 START

Invalid Matches:
M2 -> POSTED before START on field BETA

Replay Matches:
M2

Time Range: 08:00:00 to 08:20:00
=====
```

5. Bish

Program Name: Bish.java

Input File: bish.dat

During a recent field trip, you got separated from your classmates and trapped on a volcanic island. Luckily there is a shop nearby that will sell you boots to traverse the molten island and rejoin your friends, however you're not sure if you have enough money to purchase the correct boots. Each pair of boots can withstand temperatures up to K – and if you step on a piece of molten rock hotter than K then you will burn and perish. There's a handy map of the island represented by an $N \times M$ grid on the wall of the shop where each grid space is the temperature of that spot on the island, so you decide to use your laptop and write a program to calculate the minimum value K required to get off the island.

Input:

The input line will contain a single integer N ($1 \leq N \leq 100$) denoting the number of volcanoes to escape. The next N input lines will start with two integers R, C ($1 \leq R, C \leq 1000$) representing the number of rows and columns in the grid representation of the island. The following R lines will contain C integers $T_{i,j}$ ($0 \leq T_{i,j} \leq 10^9$) denoting the temperature of that cell. The next line will contain four space separated integers X_1, Y_1, X_2, Y_2 representing the starting position and the desired ending position.

Output:

For each volcanic island, print the smallest value K required to escape the island.

Sample input:

```
3
3 3
1 3 2
6 5 4
9 8 1
0 0 2 2
2 2
1 10
2 3
0 0 1 1
5 5
42 42 42 42 42
42 42 42 42 42
42 42 42 42 42
42 42 42 42 42
42 42 42 42 42
0 0 4 4
```

Sample output:

```
4
3
42
```

6. Jackie

Program Name: Jackie.java

Input File: jackie.dat

Jackie has joined the chess club. She is new to the game, and wants you to write a program to help her determine possible combinations of moves, given the pieces already on the board. You will be given all the pieces, and their positions, on an 8x8 board (1-indexed), and you need to determine how many valid combinations of moves exist for all the pieces on the board, without the pieces interfering with each other's moves. To determine move combinations, first determine all the possible moves each piece can make from their current position, given the knowledge that rooks can move right, left, up, or down up to 7 spaces, bishops can move up to 7 spaces on any of the diagonals, and a queen can do either. Each move combination is made up of 1 valid move (and finishing position on the board) per each piece, note that a valid move for a piece is to stay in its current position. Once a combo has been chosen, you will need to validate that the combo has no collisions. Collisions occur, when on the course of making your move, two or more of the pieces move into the same space at the same time. For each combo, assume you will be making all moves simultaneously, moving each piece one space at a time until all pieces are in the positions you chose when you made the combo. If two pieces move into the same space at the same time, then that causes a collision, and the move combo is no longer valid. Pieces can swap places while on their paths to the finishing positions, without causing a collision. See the example section for an explanation of some of the possible scenarios.

Example: All positions are in form row,column

We have a rook in position 3,3, and a queen in position 2, 2, as shown in the sample board below. We can choose any of the valid moves for those two pieces (rooks can go up, down, left, right, queen can do the same and also diagonals). For example, if we choose queen to 6,2 and rook to 3,6, there are no collisions, and this would make a valid move combination. If we choose to not move the rook, and move the queen to any point along the down-right diagonal, the queen cannot move as the rook will always be in the way, so none of those move combos would be valid. However, if we move the rook to 2,3, we can move the queen along the diagonal, because the rook will move off of the diagonal at the same time as we move the queen along the diagonal (shown in the second example, we did rook to 2,3 and queen to 5,5). Now, let's explain the first sample board below. We chose as our move combo, queen to 2,8 and rook to 1,3. So we try the move combo, moving the pieces one space at a time simultaneously until they both get to the finishing position, or there is a collision, signified by the X (which there is in this case at space 2,3 on the first space moved for both pieces)

.....						
.Q.....	..X.....		.Q.....	..R.....	..R.....	..R.....
..R.....			..R.....	..Q.....		
.....					..Q.....	
..... ->			 ->	 ->	 ->	Q....
.....						
.....						
.....						

Input:

The input will begin with one integer, n ($0 < n \leq 100$). Each test case will begin with a line with an integer p ($0 < p \leq 10$), denoting the number of pieces present on the 8x8 board. The following line will contain p space-separated strings ("rook", "bishop", or "queen"), denoting the type of each of the p pieces in order. The line after that will contain p integers, each denoting the row coordinate of the position of that piece, with the following line containing another p integers, denoting the column coordinate of the position of that piece.

Output:

For each test case, output the number of combinations of valid moves of all pieces on the board, excluding the options which have any collisions, following the methodology outlined in the example section.

Sample input:

```
2
1
queen
1
1
2
rook bishop
2 5
1 5
```

Sample output:

```
22
206
```

7. Bash

Program Name: Bash.java

Input File: bash.dat

Bored during class, you go over to your friends Maelle and Alicia to see if they want to play some games and find that they're busy doing some smart stuff you can't be bothered to understand. Left alone, you decide to write a simple program to play with you to have some fun. The game of choice is Ultimate Tic Tac Toe, which is like a normal game of tic tac toe but within each square is a new tic tac toe board! When a player makes a move inside a square, the next player must move in the large square corresponding to the small square that was played. For example, in the board below if a player makes a move in square 3 then the next move must be made in the top right corner of the larger board.

<table><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>										<table><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>										<table><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>									
<table><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>										<table> <tr> <td>1</td><td>2</td><td>3</td></tr> <tr> <td>4</td><td>5</td><td>6</td></tr> <tr> <td>7</td><td>8</td><td>9</td></tr> </table>	1	2	3	4	5	6	7	8	9	<table><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>									
1	2	3																											
4	5	6																											
7	8	9																											
<table><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>										<table><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>										<table><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>									

The first move always takes place on the middle board. If a move takes you to an already finished board (the board is full or has a winner) then you will choose to move on the first available board 1-9 where 1 is top left and 9 is bottom right. Both you and the robot make random moves each turn, since you can't be bothered to try and play optimally. When generating a move, number the squares 1-9 like above skipping filled in spaces, then generate a random number 1-N where N is the highest value on that board using `random.nextInt(a,b)`. The robot will always move first placing Xs and you will go second playing Os. Winning a sub-game follows normal tic tac toe rules, where the first player to make 3 in a row horizontally, vertically, or diagonally wins. The winner of the ultimate tic tac toe game is the player to win 3 sub-games horizontally, vertically, or diagonally.

Input:

The input file will contain two values, an integer N ($1 \leq N \leq 10^6$) denoting the number of games to be played and a long value L representing a random seed. The random seed provided will be used to generate a new random seed for each of the N games using `random.nextLong()`.

Output:

For each game output the winner "X wins!", "O wins!", or "Tie" and the final board state like shown below. After all games output the statistics for X, O, and Tied games to two decimal places.

Sample input:

3 235325225275745653

~ Sample output next page ~

~ *Bash continued* ~

Sample output:

```
Tie
\ / /-\ /-\
 x | | | |
 / \ \-/ \-/

/-\ XOx \ /
| | XOx x
\-/ OXO / \

\ / \ / /-\
 x x | |
 / \ / \ \-/
-----
O wins!
/-\ /-\ \ /
| | | | x
\-/ \-/ / \

/-\ \ / \ /
| | x x
\-/ / \ / \

/-\ \ / XXO
| | x ..O
\-/ / \ OOX
-----
X wins!
X.X ..X \ /
.O. .OO x
O.O XOO / \

/-\ /-\ \ /
| | | | x
\-/ \-/ / \

\ / ..O \ /
 x XX. x
 / \ ..X / \
-----
X: 33.33%
O: 33.33%
Tie: 33.33%
```

8. Steve

Program Name: Steve.java

Input File: steve.dat

Steve is trapped in a maze with his friends! He needs your help to write a program to find out how long it will take them all to escape as efficiently as possible. They have been given a flashlight and an aerial view map of the maze, denoting by the following characters:

- S – Denotes the starting point in the maze (where Steve and all of his friends begin)
- E – Denotes the end point of the maze, where all people need to end up.
- # - Denotes a wall in the maze, cannot be passed.
- . – Denotes an empty space in the maze, can be passed.

You may only move up, down, left, and right, and you can assume all sections (characters) are equal in size.

You will also be given the speeds of all the people in the maze, i.e. how long it will take to pass one section of the maze (one character). In order to escape the maze, all people must get from the start to the end, but they can only do that if they have the map and the flashlight, it is too dark and confusing otherwise. The maze is only wide enough for a maximum 2 people to travel at a time, so the process is: two people take the map and flashlight to the end, one comes back with the flashlight and map and either keeps the map/flashlight or gives it to someone else, and then two people travel to the end again, with one coming back, until all people have made it to the end. Keep in mind, when 2 people travel, they can only move as fast as the slower of the two of them.

Input:

The input will begin with one integer, n ($0 < n \leq 1000$) denoting the number of test cases to follow. Each test case will begin with a line with three space-separated integers r ($0 < r \leq 10$), c ($0 < c \leq 10$), and p ($0 < p \leq 10$), denoting the number of rows and columns in the map given, and the number of people (including Steve) who need to escape the maze, respectively. The following r lines will each contain c characters denoting the aerial view of the maze. The following line will contain p space separated integers denoting the speeds of each of the people in the maze.

Output:

For each test case, output the minimum amount of time required for all of the people in the maze to get out. If the maze is not solvable for any reason (too many walls, no end point), output the string "Phooey".

Sample input:

```
2
4 5 4
S..#.
#...#
##...
#.E.#
1 2 5 8
3 6 3
S..#.E
##...#
..#...
2 4 6
```

Sample output:

```
75
108
```

9. Bosh

Program Name: Bosh.java

Input File: bosh.dat

During math class you got bored and started drawing random shapes on your paper. However, when your math teacher saw you doing this, to punish you, they assigned you some homework. You are to calculate the approximate area of each of the shapes you drew on your paper. Luckily, it's grid paper so you have exact coordinates of the vertices on the polygon, but instead of doing the lame calculus, you decide writing a program to solve it for you is easier. Luckily from an earlier lecture you remember two possible ways to calculate the area of a polygon.

The first is called the shoelace formula, or Guass's area formula, and only works for simple polygons (which these are). For a polygon with vertices $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ and with $(x_{n+1}, y_{n+1}) = (x_1, y_1)$.

$$A = \frac{1}{2} \left| \sum_{i=1}^n (x_i y_{i+1} - x_{i+1} y_i) \right|$$

The second is a more fun method called Monte Carlo area estimation, which is a probabilistic technique that estimates the area of an irregular shape by bounding it with a known, larger area and randomly scattering points. First, you surround the polygon with a rectangle and then drop random points in it. The area can then be calculated:

$$\hat{A} = A_{\text{box}} \cdot \frac{N_{\text{inside}}}{N}$$

Where N is the number of points thrown. When calculating the estimated area, use $N = 100,000$ and set the bounds of the rectangle to the smallest value possible to encompass the polygon.

Input:

The input line will contain a single integer N ($1 \leq N \leq 100$) denoting the number of polygons drawn. The following N lines will each start with an integer V ($1 \leq V \leq 1000$) denoting the number of vertices on the polygon and a random seed R to be used for the Monte Carlo method. The following V lines of input will contain two space separated integers X, Y ($-1000 \leq X, Y \leq 1000$) denoting the coordinates of that vertex. For all test cases, $A \neq 0$.

Output:

For each polygon output the exact area using the shoelace method, the estimated area, and the percent difference between the two values as shown below.

Sample input:

```
4
4 4262421353
0 4
4 4
4 0
0 0
3 1643734744
0 6
6 0
0 0
4 7474584534
0 5
10 5
10 0
0 0
5 7213464287
3 8
8 6
9 1
1 0
0 5
```

Sample output:

```
Exact = 16.00
Estimate = 16.00
Difference = 0.00%

Exact = 18.00
Estimate = 18.07
Difference = 0.36%

Exact = 50.00
Estimate = 50.00
Difference = 0.00%

Exact = 51.50
Estimate = 51.35
Difference = 0.30%
```

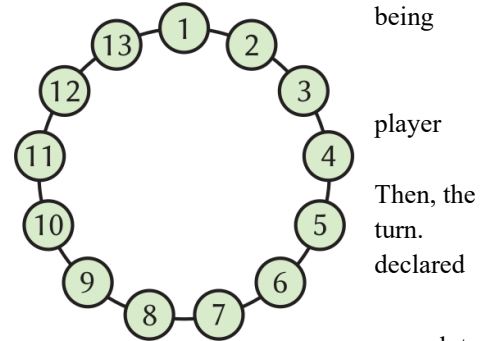
10. Gustave

Program Name: Gustave.java

Input File: gustave.dat

Gustave is sitting in a circle with several of his friends and explained the following rules of the game they are about to play:

1. Players are numbered 1 through n , with the player clockwise of player i numbered player $i + 1$ until all players have received a number.
2. Every player starts as “active”.
3. When an active player is tapped by another active player, the tapped becomes an “inactive” player.
4. Player 1 begins the game and taps the active player clockwise of them.
5. This process continues until only one player remains active, and they are the winner.



While many people might see this as a game of pure luck, Gustave is smart enough to recognize that this is a deterministic choice, and if he positions himself among his classmates intelligently, he can guarantee a win. However, Gustave is too tired and cannot figure out what position he should be in. Given the number of players n (including Gustave), determine which number he wants to be assigned so that she guarantees a win.

Input:

The first line will consist of a single integer T , $1 \leq T \leq 2.5 \cdot 10^5$, denoting the number of testcases to follow. Each of the following T lines will consist of a single integer denoting the value of n , $1 \leq n < 10^{10}$, representing the total number of players, Gustave included.

Output:

For each test case, on its own line, print the integer corresponding to the numeric position that Gustave should be sitting in to guarantee his win; for those who come after.

Sample input:

7
5
6
7
1
12
13
14

Sample output:

3
5
7
1
9
11
13

Explanation:

In the first example, player 1 taps player 2. Player 3 is the closest active player clockwise of player 2. Player 3 taps 4. Then, player 5 taps player 1. Then player 3 taps player 5, and thus Player 3 is the winner.

In the second example, we get P1 taps P2. P3 taps P4. P5 taps P6. P1 taps P3. P5 taps P1. Thus, P5 is the winner.

In the third example, we get P1 taps P2. P3 taps P4. P5 taps P6. P7 taps P1. P3 taps P5. P7 taps P3. Thus, P7 is the winner.

11. Verso

Program Name: Verso.java

Input File: verso.dat

Verso is a linguistics professor who has become obsessed with palindromes – words that read the same forwards and backwards. While grading papers, Verso noticed that any string can be broken into palindromic segments, but different ways of breaking them up require different amounts of "mental effort" to verify.

Verso has developed a cost model for splitting strings: each split at position i (creating a boundary between characters at index i and $i + 1$) costs exactly i points of mental effort. For example, splitting "RACE" into "R", "A", "CE" requires a split after position 1 (cost 1) and after position 2 (cost 2), for a total cost of 3. However, if each resulting segment isn't a palindrome, Verso gets frustrated and the solution is invalid, as would be the case with the split provided above.

Verso wants to break strings into palindromic segments while minimizing his total mental effort. Note that every single character is considered a palindrome on its own, so a valid solution always exists.

Input:

The first line will consist of a single integer T , $1 \leq T \leq 100$, denoting the number of test cases to follow. Each of the following T lines will consist of a single string S consisting of uppercase English letters only, where $1 \leq |S| \leq 10^3$.

Output:

For each test case, on its own line, print the minimum total cost to partition the string into palindromic segments. If the entire string is already a palindrome, the answer is 0 (no splits needed).

Sample input:

```
6
A
AB
ABA
RACECAR
ABC
ABCD
```

Sample output:

```
0
1
0
0
3
6
```

Explanation:

The first, third, and fourth test cases are all already palindromes, and thus their cost is 0.

The second requires a split in between "A" and "B" which costs 1. This is the minimum (and only) way to split the string, and this its minimum cost is 1.

The fifth string requires a split in between "A", "B", and "C", which costs 3 in total (1 for A|BC and 2 for B|C).

The sixth string works exactly the same as the fifth but additionally requires a split between C|D which costs 3, bringing the total to 6.

12. Maelle

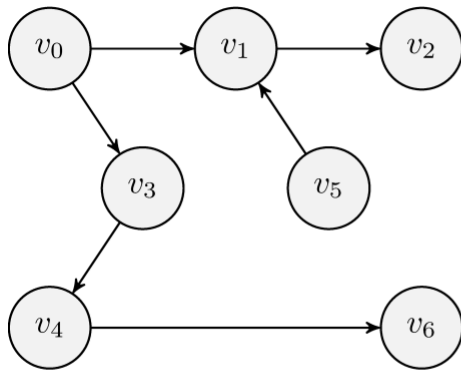
Program Name: Maelle.java

Input File: maelle.dat

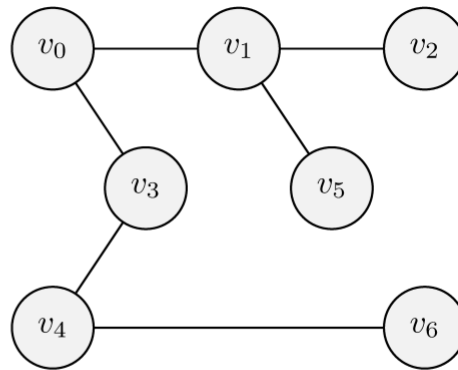
Maelle is good friends with Alicia who, earlier this year, was learning about the formal definitions of Graphs and Trees. Previously, Alicia had mentioned that she had a sage, oracle friend who had determined the connectedness of each graph ahead of time; however, as it turns out, Maelle was that friend! As a quick reminder, Alicia learned:

- A graph $G = (V, E)$ is formally defined to be composed of a set of vertices $V = \{v_1, v_2, \dots, v_n\}$ and edges $E = \{e_1, e_2, \dots, e_m\}$ where edge $e_i = (u, v)$, $u, v \in V$ indicates that a relationship exists between vertex u and vertex v .
- A graph G is said to be “connected” if every pair of vertices v_i, v_j is connected in G . A pair of vertices v_i, v_j is said to be connected if there exists some simple path from v_i to v_j (denoted $v_i \rightsquigarrow v_j$) via some subset of edges in E .
 - An *undirected* graph G is said to be “disconnected” if it is not connected.
 - A *directed* graph G is said to be “weakly connected” if by replacing all directed edges in E with undirected ones (call this new set E'), then graph $G' = (V, E')$ is connected.
 - A *directed* graph G is said to be “strongly connected” if for every pair of vertices $v_i, v_j \in V$, then *both* $v_i \rightsquigarrow v_j$ and $v_j \rightsquigarrow v_i$ hold. That is, a graph is strongly connected if it is a connected directed graph.
 - A *directed* graph G is said to be “unilaterally connected” if, for every pair of vertices $v_i, v_j \in V$, then *either* $v_i \rightsquigarrow v_j$ and/or $v_j \rightsquigarrow v_i$ hold.
 - A *directed* graph G is said to be “disconnected” if it is not weakly, strongly, or unilaterally connected.

Unfortunately, Maelle has recently lost her sage powers and is unable to automatically classify the connectedness of each graph anymore. In order to maintain her reputation, help Maelle in determining the connectedness of Alicia’s graphs.



Directed, Weakly Connected, Acyclic Graph



Tree (Undirected, Connected, Acyclic Graph)

Input:

The first line of input will consist of a single integer T , $1 \leq T \leq 15$, denoting the number of test cases to follow. The next T test cases will each consist of a single line denoting the serialized string of the object consisting of the set of vertices and edges of the graph. Each graph object can be described by two integers n and m , $2 \leq n \leq 10^3$, $1 \leq m \leq n \cdot (n - 1)$ denoting the number of vertices and edges. You may assume that graph G has $V = \{v_0, v_1, \dots, v_{n-1}\}$ for the specified value of n , and each edge $e = (v_i, v_j) \in E$ will have $0 \leq i, j < n$, $i \neq j$ hold. You may also assume that both the `directedness` and `cyclicity` fields have been filled out with the corresponding `Directedness` and `Cyclicity` enum values.

Output:

For each test case, on its own line, output a single string in the form of “<directedness>, <connectivity>, <cyclic> graph”. If the graph is also a tree, start the line off with the word “Tree ”, and include the other output for this line in parentheses. As a reminder, a tree is defined to be an undirected, connected, acyclic graph.

~ Maelle continued ~

Sample input: (lines truncated at ...)

5

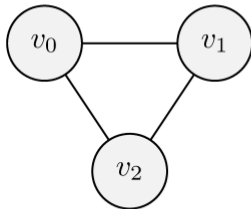
```
r00ABXNyAAVHcmFwaAAAAAAAAAABAgAGSQABbUkAAW5MAA1jb25uZWNOZW RuZXNzdAAPTENvbm5lY3R...
r00ABXNyAAVHcmFwaAAAAAAAAAABAgAGSQABbUkAAW5MAA1jb25uZWNOZW RuZXNzdAAPTENvbm5lY3R...
r00ABXNyAAVHcmFwaAAAAAAAAAABAgAGSQABbUkAAW5MAA1jb25uZWNOZW RuZXNzdAAPTENvbm5lY3R...
r00ABXNyAAVHcmFwaAAAAAAAAAABAgAGSQABbUkAAW5MAA1jb25uZWNOZW RuZXNzdAAPTENvbm5lY3R...
r00ABXNyAAVHcmFwaAAAAAAAAAABAgAGSQABbUkAAW5MAA1jb25uZWNOZW RuZXNzdAAPTENvbm5lY3R...
```

Sample output:

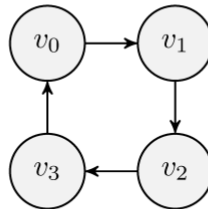
```
Directed, Weakly Connected, Acyclic Graph
Tree (Undirected, Connected, Acyclic Graph)
Undirected, Connected, Cyclic Graph
Directed, Strongly Connected, Cyclic Graph
Directed, Unilaterally Connected, Acyclic Graph
```

Explanation:

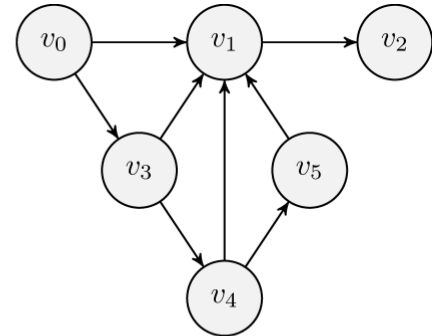
The first test case corresponds to the graph in the figure on the left, and the second test case corresponds to the graph in the figure on the right. The three additional test cases have been illustrated below.



Undirected, Connected, Cyclic Graph



Directed, Strongly Connected, Cyclic Graph



Directed, Unilaterally Connected, Acyclic Graph

Important Notes:

There are many factors that affect how object serialization works that don't pertain to what the object logically represents. Because of this, you should avoid changing the names of existing methods/fields, creating additional `public`/`protected` methods (`private` is fine) and avoid creating any new fields as this affects what data is associated with the encoding of the object. You may set the `toString()` method to simply return the empty string if you do not wish to make use of it in your implementation.

Starter Class:

```
import java.io.Serializable;
import java.util.HashMap;
import java.util.HashSet;

enum Directedness {
    Undirected, Directed;
}

enum Cyclicity {
    Cyclic, Acyclic;
}

enum Connectedness {
    Connected, Disconnected, Weakly_Connected, Strongly_Connected, Unilaterally_Connected;
}

class Graph implements Serializable {
    public static final long serialVersionUID = 1L;

    private HashMap<Integer, HashSet<Integer>> g;
    private int n, m;

    public Graph(HashMap<Integer, HashSet<Integer>> g, int n, int m) {
        this.g = g;
        this.n = n;
        this.m = m;
    }

    protected Directedness directedness;
    protected Cyclicity cyclicity;
    protected Connectedness connectedness;

    public Connectedness getConnectedness() {
        // TODO: Implement
    }

    @Override
    public String toString() {
        // TODO: Implement
    }
}
```


Serialization Code:

```
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
import java.util.Base64;

class Serializer {
    public static String marshal(Serializable o) throws IOException {
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        ObjectOutputStream oos = new ObjectOutputStream(baos);
        oos.writeObject(o);
        String encodedData = Base64.getEncoder().encodeToString(baos.toByteArray());
        baos.close();
        oos.close();
        return encodedData;
    }

    public static Object unmarshall(String s) throws ClassNotFoundException, IOException {
        byte[] data = Base64.getDecoder().decode(s);
        ByteArrayInputStream bais = new ByteArrayInputStream(data);
        ObjectInputStream ois = new ObjectInputStream(bais);
        Object o = ois.readObject();
        bais.close();
        ois.close();
        return o;
    }
}
```

Sample Usage:

```
String byteStream = file.readLine();
Object o = null;
try {
    o = Serializer.unmarshall(byteStream);
} catch (Exception e) {
    System.out.println("Come on problem setter, get your classes straight!");
    return;
}

Graph g = null;
if (o instanceof Graph) {
    g = Graph.class.cast(o);
} else {
    System.out.println("Come on problem setter, get your classes straight!");
    return;
}
```